Dry run Sieve algorithm

Let's understand this code **step by step**, because there are three main parts:

1. **Sieve of Eratosthenes** (find all prime numbers)
2. **Check whether a number is a Fear Prime**
3. **Prefix Sum** (answer each query in O(1))

What is a Fear Prime?

A number is called a **Fear Prime** if:

- It is prime.
- It does **not contain digit 0**.
- If we repeatedly remove digits from the **left**, every remaining number is also prime.

Example:

```
313

313 → Prime ✅
13  → Prime ✅
3   → Prime ✅

Therefore 313 is a Fear Prime.
```

Example:

```
233

233 → Prime ✅
33  → Not Prime ❌

Not a Fear Prime.
```

Step 1 : Global Variables

```
C++
const int MAXN = 1000000;
```

Maximum possible number.

```
C++
vector<bool> isPrime(MAXN + 1, true);
```

Stores whether a number is prime.

Initially

```
0 -> true
1 -> true
2 -> true
3 -> true
...
1000000 -> true
```

Later sieve will correct these values.

```
C++
vector<int> prefix(MAXN + 1, 0);
```

Stores

```
prefix[i]
=
number of fear primes
≤ i
```

Example later

```
prefix[100]=18

means
```

```
18 fear primes exist from
1...100
```

Step 2 : Sieve

```
C++
void sieve()
```

Purpose:

Find all primes up to 1,000,000.

Initially

```
2 3 4 5 6 7 8 9 10 ...

all marked prime
```

except

```
0
1
```

First line

```
C++
isPrime[0]=isPrime[1]=false;
```

Now

```
0 ❌
1 ❌
2 ✅
3 ✅
4 ✅
5 ✅
...
```

Outer loop

```
C++
for(int i=2;i*i<=MAXN;i++)
```

Only go till √MAXN.

Reason:

Every composite number has one factor ≤ √n.

For

```
100

√100 =10

Need only check till 10.
```

Suppose

```
i=2
```

Since

```
2 is prime
```

remove all multiples.

```
C++
for(int j=i*i;j<=MAXN;j+=i)
```

Starts from

```
4
```

instead of

```
2
```

because

```
2 already known prime

2×2=4 first composite
```

Removes

```
4
6
8
10
12
14
...
```

Next

```
i=3
```

Remove

```
9
12
15
18
21
...
```

Next

```
i=4
```

Already marked composite.

Skip.

Next

```
i=5
```

Remove

```
25
30
35
40...
```

Continue.

Finally

```
isPrime[x]

true → prime

false → composite
```

Step 3 : Fear Prime Function

```
C++
bool isFearPrime(int x)
```

Suppose

```
x=313
```

Check 1

```
C++
if(!isPrime[x]) return false;
```

313

Prime

Continue.

Convert to string

```
C++
string s=to_string(x);
```

Now

```
s="313"
```

Check zero

```
C++
for(char c:s)
    if(c=='0')
        return false;
```

Digits

```
3
1
3
```

No zero.

Continue.

Check every suffix

Loop

```
C++
for(int i=1;i<s.size();i++)
```

Notice

Starts from

```
1
```

because

Entire number already checked.

First iteration

```
i=1
```

Substring

```
C++
s.substr(1)
```

```
13
```

Convert

```
C++
stoi("13")
```

```
13
```

Prime?

Yes.

Continue.

Second iteration

```
i=2
```

Substring

```
3
```

Prime?

Yes.

Done.

Return

```
true
```

Example 2

Suppose

```
x=233
```

```
233
```

Prime

No zero.

Suffix

```
33
```

Prime?

No.

Immediately

```
return false;
```

Step 4 : Building Prefix Array

```
C++
for(int i=1;i<=MAXN;i++)
```

Go through every number.

Initially

```
prefix[0]=0
```

Suppose numbers are

```
1
2
3
4
5
6
7
...
```

i=1

```
prefix[1]=prefix[0]

0
```

Fear Prime?

No.

```
prefix[1]=0
```

i=2

```
prefix[2]=0
```

Fear prime?

```
2

Prime
No zero
No suffix

YES
```

Increase.

```
prefix[2]=1
```

i=3

Fear prime

Yes.

```
prefix[3]=2
```

i=4

Not prime

```
prefix[4]=2
```

i=5

Fear prime

```
prefix[5]=3
```

i=13

```
13

Prime

Suffix

3

Prime
```

Fear prime

```
prefix[13]=5
```

(assuming previous count 4)

Eventually

```
prefix[100]
```

stores

```
count of all fear primes ≤100
```

Step 5 : Answer Queries

Suppose

Input

```
3

10
20
100
```

Need

```
Fear primes ≤10

Fear primes ≤20
```

```
Fear primes ≤100
```

Instead of recalculating every time,

Simply do

```
C++
cout<<prefix[N];
```

Each query

```
O(1)
```

Complete Dry Run

Input

```
1
20
```

Step 1

Run sieve.

Primes

```
2
3
5
7
11
13
17
19
...
```

Step 2

Check every number

```
2
```

Fear prime?

```
Prime

No zero

No suffix

YES
```

Count=1

```
3
```

Fear prime

Count=2

```
5
```

Fear prime

Count=3

```
7
```

Fear prime

Count=4

```
11
```

Suffix

```
1
```

Not prime

Not fear prime

```
13
```

Suffix

```
3
Prime
```

Fear prime

Count=5

```
17
```

Suffix

```
7
Prime
```

Fear prime

Count=6

```
19
```

Suffix

```
9
Not prime
```

Not fear prime.

Now

```
prefix[20]=6
```

Output

```
6
```

Fear primes ≤20 are:

```
2
3
5
7
13
17
```

Time Complexity

Part	Complexity
Sieve	**O(N log log N)**
Checking all numbers for Fear Prime	**O(N × digits) ≈ O(6N)** since numbers ≤ 1,000,000 have at most 7 digits
Building Prefix Array	Included in above loop, overall **O(N)**
Each Query	**O(1)**

So the total preprocessing is approximately **O(N log log N + N)**, and after that every query is answered in constant time using the `prefix` array.

Let's derive the **time complexity of the Sieve of Eratosthenes** mathematically, step by step.

Code

```
C++
```

```
for (int i = 2; i * i <= n; i++) {
    if (isPrime[i]) {
        for (int j = i * i; j <= n; j += i) {
            isPrime[j] = false;
        }
    }
}
```

There are **two loops**, so we'll analyze each one.

Step 1: Outer Loop

```
C++
for (int i = 2; i * i <= n; i++)
```

Condition is

```
C++
i * i <= n
```

Taking square root,

```
i <= √n
```

Therefore the outer loop runs

```
√n times
```

So,

$$T_{outer} = O(\sqrt{n})$$

But this **is not** the final complexity because the inner loop does different amounts of work for each `i`.

Step 2: Inner Loop

Suppose

```
C++
i = 2
```

Then

```
C++
for(j = 4; j <= n; j += 2)
```

The values are

```
4
6
8
10
...
n
```

Number of iterations is approximately

$$\frac{n}{2}$$

Now suppose

```
C++
i = 3
```

Loop is

```
C++
j = 9,12,15,...
```

Iterations

$$\frac{n}{3}$$

For

```
C++
i = 5
```

Iterations

$$\frac{n}{5}$$

For

```
C++
i = 7
```

Iterations

$$\frac{n}{7}$$

Generally,

for prime p,

iterations are approximately

$$\frac{n}{p}$$

Step 3: Add the Work

Ignoring constants,

Total work becomes

$$\frac{n}{2} + \frac{n}{3} + \frac{n}{5} + \frac{n}{7} + \cdots$$

Factor out n,

$$n\left(\frac{1}{2} + \frac{1}{3} + \frac{1}{5} + \frac{1}{7} + \cdots\right)$$

Now the question becomes

> What is

$$\frac{1}{2} + \frac{1}{3} + \frac{1}{5} + \frac{1}{7} + \cdots$$

where the sum is over **primes**?

Step 4: Prime Harmonic Series

A famous result in number theory says

$$\sum_{p \le n} \frac{1}{p} = \log\log n + O(1)$$

This means

$$\frac{1}{2} + \frac{1}{3} + \frac{1}{5} + \cdots = \log\log n$$

approximately.

Step 5: Multiply Back

Therefore

$$T(n) = n \cdot \log\log n$$

Hence

$$\boxed{O(n \log \log n)}$$

Why not $O(n\sqrt{n})$?

A common mistake is saying:

- Outer loop runs √n times.
- Inner loop runs n/i times.
- Therefore,

$$\sqrt{n} \times n = n\sqrt{n}$$

This is **incorrect** because the inner loop **does not always run n times**. It runs:

i	Iterations
2	n/2
3	n/3
5	n/5
7	n/7
11	n/11
13	n/13

The number of iterations keeps decreasing as `i` increases, so you **sum** the costs instead of multiplying them.

Example for $n = 100$

The primes up to $\sqrt{100} = 10$ are:

```
2, 3, 5, 7
```

Work done:

Prime	Iterations
2	100/2 = 50
3	100/3 ≈ 33
5	100/5 = 20
7	100/7 ≈ 14

Total

```
50 + 33 + 20 + 14 = 117
```

This matches

$$100 \left(\frac{1}{2} + \frac{1}{3} + \frac{1}{5} + \frac{1}{7} \right) \approx 117$$

Notice it's **much smaller** than

```
100 × √100 = 1000
```

which is why the complexity is **not** $O(n\sqrt{n})$.

Intuition

The sieve is fast because:

1. **Only prime numbers** perform marking. Composite numbers are skipped due to `if (isPrime[i])`.
2. **Each prime marks only its multiples**, which is about n/p operations.
3. The sum of the reciprocals of the primes grows extremely slowly, like $\log \log n$, leading to the final complexity:

$$\boxed{\text{Time Complexity } = O(n \log \log n)}$$

and the space complexity is:

$$\boxed{O(n)}$$

This is a very good question. The confusion is:

> **If the outer loop runs only $\sqrt{n}$ times, why does the final complexity become $O(n \log \log n)$ instead of $O(\sqrt{n} \log \log n)$?**

The answer is **because the dominant work is done by the inner loop**, not the outer loop.

Let's understand it carefully.

Step 1: Outer loop

```
C++
for(int i = 2; i * i <= n; i++)
```

The number of iterations is

$$\sqrt{n}$$

For example, if

```
n = 100
```

then

```
i = 2,3,4,5,6,7,8,9,10
```

Only **9 iterations**.

So the outer loop itself costs

$$O(\sqrt{n})$$

Step 2: Inner loop

Suppose

```
n = 100
```

When

```
i = 2
```

the inner loop runs

```
4
6
8
...
100
```

≈ 50 iterations

When

```
i = 3
```

it runs

```
9
12
15
...
99
```

≈ 33 iterations

When

```
i = 5
```

≈ 20 iterations

When

```
i = 7
```

≈ 14 iterations

Total work is

```
50 + 33 + 20 + 14
```

Notice something important:

The **inner loop alone has already done 117 operations**, while the outer loop only executed **9 times**.

The outer loop is almost negligible compared to the work done inside it.

Step 3: Write the sum

The total work is

$$\frac{n}{2} + \frac{n}{3} + \frac{n}{5} + \frac{n}{7} + \cdots$$

Factor out n:

$$n\left(\frac{1}{2} + \frac{1}{3} + \frac{1}{5} + \frac{1}{7} + \cdots\right)$$

The remaining sum is

$$\log\log n$$

So

$$T(n) = n \log\log n$$

Why don't we multiply by $\sqrt{n}$?

Many beginners think:

```
Outer loop = √n
Inner loop = n/i
Answer = √n × n
```

This is **incorrect**.

Why?

Because the inner loop **doesn't run n/i for every number from 2 to $\sqrt{n}$** in the same amount. Each iteration has a **different cost**.

Think of it like this:

i	Inner loop work
2	50
3	33
5	20
7	14

You **add** these costs:

```
50 + 33 + 20 + 14
```

You do **not** multiply:

```
4 × 50
```

because each outer-loop iteration performs a different amount of work.

General Rule for Nested Loops

If the inner loop always runs the same number of times:

```C++
for(i=1;i<=n;i++)
    for(j=1;j<=n;j++)
```

then

$$O(n) \times O(n) = O(n^2)$$

because every outer iteration does exactly n inner iterations.

But if the inner loop depends on `i`:

```C++
for(i=1;i<=n;i++)
    for(j=i;j<=n;j++)
```

the work is

$$(n) + (n-1) + (n-2) + \cdots + 1$$

which is a **sum**, not a multiplication.

Similarly, in the sieve:

```C++
for(i=2;i*i<=n;i++)
```

the work is

$$\frac{n}{2} + \frac{n}{3} + \frac{n}{5} + \cdots$$

which simplifies to

$$\boxed{O(n \log \log n)}$$

Key takeaway

- The **outer loop** only determines **how many different inner loops** are executed ($\sqrt{n}$ of them).
- The **total running time** is the **sum of the work done by all those inner loops**, not $\sqrt{n}$ multiplied by the maximum inner-loop cost.
- Since the sum of the inner-loop work is $n \log \log n$, that term dominates the $\sqrt{n}$ cost of the outer loop.

So the final complexity is:

$$\boxed{O(n \log \log n)}$$

not

$$O(\sqrt{n} \cdot n).$$